Spring Core

Interview Preparation Guide • 2.5+ YOE Level

This guide covers Spring Core end-to-end: IoC/DI, beans, scopes, lifecycle, annotations, auto-configuration, profiles, config binding, circular dependencies, and AOP. Each concept includes a plain explanation, an analogy, and an interview-ready line.

1. Spring vs Spring Boot

Spring is a Java framework built around Dependency Injection, but it needs heavy configuration and manual server setup. Spring Boot sits on top and removes that boilerplate through auto-configuration, starter dependencies, and an embedded server.

Interview line: Spring Boot is an opinionated layer over Spring that reduces boilerplate via auto-configuration, starters, and an embedded server, letting you run an app with **java -jar**.

2. IoC & Dependency Injection (most asked)

Inversion of Control (IoC): you don't create objects yourself; the Spring container creates and manages them. Control is inverted from you to Spring.

Dependency Injection (DI): the container supplies (injects) the dependencies an object needs, instead of the object building them itself.

```
@Service
class OrderService {
    private final PaymentService payment;
    OrderService(PaymentService payment) {  // constructor injection
        this.payment = payment;
    }
}
```

Three types of injection:

- **Constructor** (preferred) — immutable, testable, required deps
- **Setter** — for optional dependencies
- **Field** (@Autowired on field) — avoid: hard to test

Analogy: DI is like ordering at a restaurant. You don't cook — you declare what you need and the kitchen (Spring) delivers it ready.

Interview line: I prefer **constructor injection** because it lets me mark fields **final**, guarantees required dependencies exist at creation, and makes unit testing easy since I can pass in a fake.

3. Beans & the IoC Container

A **bean** is any object the Spring container creates, manages, and injects. The **IoC container** (Application Context) is the factory that holds all beans.

Stereotype annotations (all create beans, but label the role):

- **@Component** — generic bean
- **@Service** — business logic layer

- **@Repository** — data access layer (also translates DB exceptions)
- **@Controller** / **@RestController** — web layer (RestController returns JSON)

> **Analogy:** Stereotypes are like job titles — Engineer, Accountant, Receptionist are all employees (beans), but the title tells you their role.

4. Bean Scopes & Thread Safety

Scope	Meaning
singleton (default)	One shared instance for the whole application
prototype	A brand-new instance every time it's requested
request	One instance per HTTP request (web)
session	One instance per user session (web)

> **Interview line:** Are Spring beans thread-safe? **No.** Singleton beans are shared across all threads, so they aren't thread-safe if they hold mutable state. The fix is to keep beans **stateless** — don't store request-specific data in fields.

5. Bean Lifecycle

Beans are created, used, then destroyed. Spring lets you hook into two moments:

```
@PostConstruct   // runs once, right after creation + injection
void init() { ... }

@PreDestroy      // runs once, just before shutdown
void cleanup() { ... }
```

Order: constructor → inject dependencies → @PostConstruct → bean in use → @PreDestroy → destroyed.

> **Analogy:** @PostConstruct is setting up your desk when you arrive; @PreDestroy is tidying up before you leave.

6. @Qualifier & @Primary

When multiple beans of the same type exist, Spring can't decide which to inject. Resolve it two ways:

- **@Primary** — marks one bean as the default choice
- **@Qualifier("name")** — picks a specific bean by name

> **Interview line:** When multiple beans of the same type exist, I use **@Primary** to set a default or **@Qualifier** to pick a specific one by name. @Qualifier wins over @Primary when both are present.

7. @Configuration + @Bean

Component scanning works for your own classes. But for a third-party class you can't annotate, create the bean manually in a config class:

```
@Configuration
class AppConfig {
    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();  // class you don't own
    }
}
```

Interview line: @Component is for classes I own — Spring auto-detects them. @Bean is for programmatic creation, typically third-party classes I can't annotate, defined inside a @Configuration class.

8. @SpringBootApplication & Auto-Configuration

@SpringBootApplication bundles three annotations:

- **@Configuration** — this class can define beans
- **@ComponentScan** — find and manage @Component/@Service/etc.
- **@EnableAutoConfiguration** — auto-configure based on the classpath

Auto-configuration: Spring Boot inspects the libraries on the classpath and configures matching beans automatically, using @Conditional annotations. It backs off if you define your own bean (@ConditionalOnMissingBean).

Classpath = the set of all classes and library JARs available to your program at runtime. Dependencies in pom.xml get downloaded as JARs and placed on the classpath — that's what Spring Boot inspects.

Analogy: Auto-config is a move-in-ready furnished apartment: furniture is pre-placed, but if you bring your own sofa, they back off and use yours.

Interview line: Auto-configuration inspects the classpath and configures matching beans — e.g. seeing the web starter sets up embedded Tomcat, the DispatcherServlet, and JSON. It works via @Conditional annotations and backs off if I define my own bean.

9. Startup Flow (what happens on run())

Interview line: When a Spring Boot app starts, it begins with **SpringApplication.run()**. It creates the **application context** (IoC container), does **component scanning** to find beans, runs **auto-configuration** based on the classpath, then **creates and injects** all beans. If it's a web app, it starts **embedded Tomcat on port 8080** and is ready to serve requests.

One line: main() → run() → create context → component scan → auto-configure → create & inject beans → start server → ready

10. Profiles (environment config)

Different environments (dev, QA, prod) need different settings. Profiles swap in environment-specific config based on which is active.

```
application.properties          # shared
application-dev.properties      # dev only
application-prod.properties     # prod only
```

```
spring.profiles.active=dev      # pick active profile
```

You can also make a bean exist only in certain profiles with **@Profile("dev")**.

> **Interview line:** We use profiles for environment-specific config — separate application-{env}.properties files activated via spring.profiles.active. I've used @Profile to load different beans, like separate datasource config per environment.

11. @Value vs @ConfigurationProperties

@Value — inject a single property value:

```
@Value("${app.name}")
private String appName;
```

@ConfigurationProperties — bind a whole group of related properties (type-safe):

```
@Component
@ConfigurationProperties(prefix = "app")
class AppProperties {
    private String name;
    private int maxRetries;
    // getters & setters
}
```

> **Interview line:** For a single value I use @Value, but for a group of related properties I prefer @ConfigurationProperties — it's type-safe, cleaner, supports validation, and binds a whole group at once.

12. Circular Dependencies

Bean A needs B, and B needs A. With constructor injection, Spring can't resolve this and fails at startup (BeanCurrentlyInCreationException).

Fixes:

- **Refactor** (best) — extract shared logic into a third class
- **@Lazy** — delay creating one bean until first used
- **Setter injection** — create both first, then wire

> **Interview line:** A circular dependency means two beans depend on each other; with constructor injection it fails at startup. The right fix is usually to **refactor** — extract the shared responsibility into a third class. @Lazy or setter injection are quick workarounds, but I treat it as a design smell to fix, not patch.

13. Spring AOP

AOP (Aspect-Oriented Programming) handles **cross-cutting concerns** — logging, security, transactions — logic needed across many methods but separate from business logic. Write it once, apply it automatically.

Key terms: Aspect (the logic), Advice (when: before/after/around), Pointcut (which methods), Join point (a specific application point).

```
@Service
class OrderService {
    @Transactional
    public void placeOrder() {
        // rolls back automatically on error
    }
}
```

Analogy: AOP is like security cameras — instead of a guard in every room (business logic), cameras (an aspect) monitor all rooms automatically.

Interview line: AOP handles cross-cutting concerns like logging and transactions. The classic example is **@Transactional**: Spring creates an AOP **proxy** around the bean that starts a transaction before the method and commits or rolls back after — without me writing that code.

Final Tip for 2.5 YOE

Definitions alone aren't enough at your level. For each concept, prepare a one-line **project story**: "In my project we used profiles for...", "We hit a circular dependency when...". Tying concepts to real experience is what separates memorized theory from an experienced engineer in the interviewer's mind.